Practical No 01

Aim : Implement Feed-forward Neural Network and train the network with different optimizers and compare the results

- **Description**

A Feed-Forward Neural Network (FFN) passes input data in one direction through hidden layers to produce output. In this practical, an FFN is trained on the MNIST dataset using different optimizers to compare performance.

- **Process**

1. Load the dataset.
2. Normalize pixel values and flatten 28×28 images into 784-dimensional vectors.
3. Build a Feed-Forward Neural Network with two hidden layers using ReLU activation and a softmax output layer.
4. Compile the model using categorical cross-entropy loss and accuracy metric.
5. Train the network separately using SGD, RMSProp, and Adam optimizers.
6. Evaluate each trained model on test data.
7. Compare training loss, validation accuracy, and test accuracy.

- **Optimizers description**

SGD: Simple optimizer with slower convergence.
RMSProp: Adapts learning rate for faster training.
Adam: Combines momentum and adaptive learning rates for best performance.

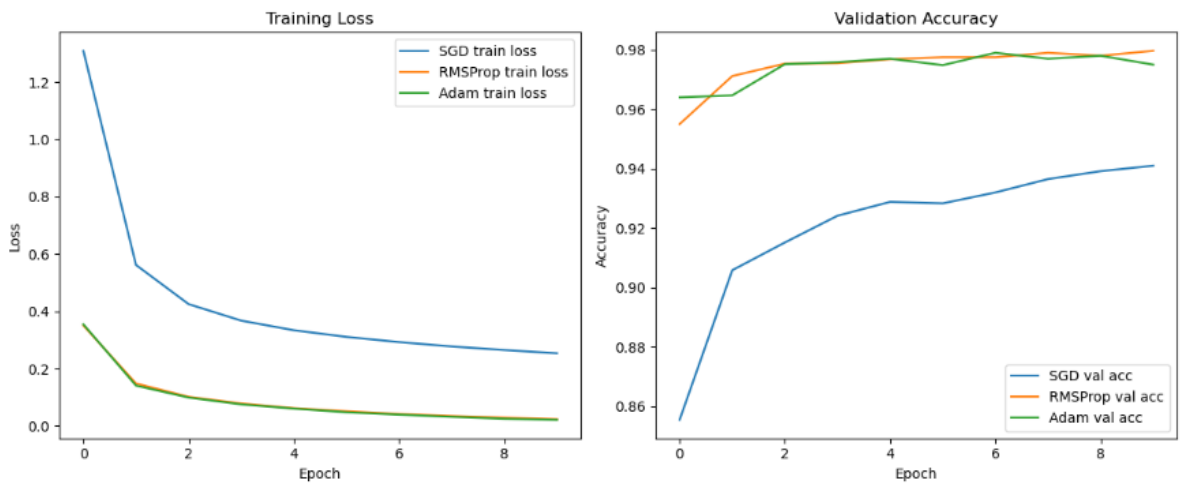
Code :

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt


(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()


x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train_flat = x_train.reshape(-1, 28*28) # shape: (60000, 784)
x_test_flat = x_test.reshape(-1, 28*28) # shape: (10000, 784)


def build_ffn():
 model = tf.keras.Sequential([
 tf.keras.layers.Input(shape=(784,)),
 tf.keras.layers.Dense(128, activation='relu'),
 tf.keras.layers.Dense(64, activation='relu'),
 tf.keras.layers.Dense(10, activation='softmax') # 10 classes
 ])
 return model
```

```
batch_size = 128
epochs = 10
loss_fn = 'sparse_categorical_crossentropy' # labels are integers 0..9
metrics = ['accuracy']


optimizers = {
 'SGD': tf.keras.optimizers.SGD(learning_rate=0.01),
 'RMSProp': tf.keras.optimizers.RMSprop(learning_rate=0.001),
 'Adam': tf.keras.optimizers.Adam(learning_rate=0.001)
}


histories = {}
for name, opt in optimizers.items():
 print(f"\n--- Training with optimizer: {name} ---")
 model = build_ffn()
 model.compile(optimizer=opt, loss=loss_fn, metrics=metrics)
 history = model.fit(
 x_train_flat, y_train,
 validation_split=0.1, # use 10% of training set for validation
 epochs=epochs,
 batch_size=batch_size,
 verbose=2
 )
 # evaluate on the test set
 test_loss, test_acc = model.evaluate(x_test_flat, y_test, verbose=0)
 print(f"{name} test accuracy: {test_acc:.4f}, test loss: {test_loss:.4f}")
 histories[name] = {
 'history': history.history,
 'test_loss': test_loss,
 'test_acc': test_acc
 }


plt.figure(figsize=(12,5))
# Training loss (per epoch)
plt.subplot(1,2,1)
for name, data in histories.items():
 plt.plot(data['history']['loss'], label=f"{name} train loss")
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
# Validation accuracy (per epoch)
plt.subplot(1,2,2)
for name, data in histories.items():
 plt.plot(data['history']['val_accuracy'], label=f"{name} val acc")
plt.title("Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
```

```
plt.legend()
plt.tight_layout()
plt.show()
```

OUTPUT:-




```
# 8. Print final test accuracies
print("\nFinal test accuracies:")
for name, data in histories.items():
 print(f"{name}: accuracy={data['test_acc']:.4f}, loss={data['test_loss']:.4f}")
```

```
Final test accuracies:
SGD: accuracy=0.9304, loss=0.2390
RMSProp: accuracy=0.9783, loss=0.0790
Adam: accuracy=0.9729, loss=0.0928
```

Result:

Adam achieved the highest accuracy and fastest convergence. RMSProp performed better than SGD. Adam was the most effective optimizer for this task.

Practical No 2

Aim:- Write a Program to implement regularization to prevent the model from overfitting

1. Aim of the Practical
To understand overfitting in neural networks and
To implement regularization techniques to improve model generalization.

2. What is Overfitting? (Explain First – Very Important)
Simple Definition

Overfitting happens when:
Model learns training data too well But performs poorly on new (test) data Analogy for Students Like memorizing exam answers without understanding concepts.

3. What is Regularization?
Definition

Regularization is a technique used to: Reduce overfitting Improve performance on unseen data

Types Used in This Practical
L2 Regularization Dropout

4. Dataset Used
We will use a simple dataset (MNIST) for clarity.
Images: Handwritten digits (0–9)
Task: Digit classification

Code:-

```
import tensorflow as tf
from tensorflow.keras import layers, models, regularizers
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt

# Step 2: Load the Dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datas
s/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 4s 0us/step


# Step 3: Preprocess the Data
x_train = x_train.reshape(60000, 28*28) / 255.0
x_test = x_test.reshape(10000, 28*28) / 255.0
```

```
# Step 4: Build a Model WITHOUT Regularization
model_no_reg = models.Sequential([
 layers.Dense(256, activation='relu', input_shape=(784,)),
 layers.Dense(256, activation='relu'),
 layers.Dense(10, activation='softmax')
])
```

```
C:\Users\Admin\AppData\Roaming\Python\Python39\site-packages\keras\src\layers\core\dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` a
rgument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
# Step 5: Compile the Model
model_no_reg.compile(
 optimizer='adam',
 loss='sparse_categorical_crossentropy',
 metrics=['accuracy']
)
```

```
# Step 6: Train the Model
history_no_reg = model_no_reg.fit(
 x_train, y_train,
 epochs=20,
 validation_data=(x_test, y_test)
)
```

```
Epoch 1/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.8988 - loss: 0.3387 - val_accuracy: 0.9667 - val_loss: 0.1094
Epoch 2/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.9722 - loss: 0.0856 - val_accuracy: 0.9749 - val_loss: 0.0790
Epoch 3/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9815 - loss: 0.0587 - val_accuracy: 0.9795 - val_loss: 0.0687
Epoch 4/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9868 - loss: 0.0409 - val_accuracy: 0.9752 - val_loss: 0.0890
Epoch 5/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9898 - loss: 0.0306 - val_accuracy: 0.9777 - val_loss: 0.0843
Epoch 6/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9919 - loss: 0.0247 - val_accuracy: 0.9801 - val_loss: 0.0819
Epoch 7/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9933 - loss: 0.0206 - val_accuracy: 0.9788 - val_loss: 0.0874
```

```
# Step 7: Build Model WITH L2 Regularization
model_l2 = models.Sequential([
 layers.Dense(256, activation='relu',
 kernel_regularizer=regularizers.l2(0.001),
 input_shape=(784,)),
 layers.Dense(256, activation='relu',
 kernel_regularizer=regularizers.l2(0.001)),
 layers.Dense(10, activation='softmax')
])
# Step 8: Compile and Train L2 Model
model_l2.compile(
 optimizer='adam',
 loss='sparse_categorical_crossentropy',
 metrics=['accuracy']
```

```
)
history_l2 = model_l2.fit(
 x_train, y_train,
 epochs=20,
 validation_data=(x_test, y_test)
)
```

```
Epoch 1/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.8894 - loss: 0.7065 - val_accuracy: 0.9523 - val_loss: 0.2983
Epoch 2/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9604 - loss: 0.2697 - val_accuracy: 0.9533 - val_loss: 0.2653
Epoch 3/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9672 - loss: 0.2203 - val_accuracy: 0.9654 - val_loss: 0.2157
Epoch 4/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9709 - loss: 0.1945 - val_accuracy: 0.9723 - val_loss: 0.1821
Epoch 5/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9725 - loss: 0.1808 - val_accuracy: 0.9688 - val_loss: 0.1834
Epoch 6/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9733 - loss: 0.1721 - val_accuracy: 0.9699 - val_loss: 0.1778
Epoch 7/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9748 - loss: 0.1630 - val_accuracy: 0.9744 - val_loss: 0.1649
Epoch 8/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9745 - loss: 0.1608 - val_accuracy: 0.9737 - val_loss: 0.1647
Epoch 9/20
```

```
# Step 9: Add Dropout Regularization
model_dropout = models.Sequential([
 layers.Dense(256, activation='relu', input_shape=(784,)),
 layers.Dropout(0.5),
 layers.Dense(256, activation='relu'),
 layers.Dropout(0.5),
 layers.Dense(10, activation='softmax')
])


# Step 10: Compile and Train Dropout Model
model_dropout.compile(
 optimizer='adam',
 loss='sparse_categorical_crossentropy',
 metrics=['accuracy']
)
history_dropout = model_dropout.fit(
 x_train, y_train,
 epochs=20,
 validation_data=(x_test, y_test)
)
```

```
Epoch 1/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.8092 - loss: 0.6048 - val_accuracy: 0.9591 - val_loss: 0.1324
Epoch 2/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9373 - loss: 0.2101 - val_accuracy: 0.9690 - val_loss: 0.1038
Epoch 3/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9509 - loss: 0.1674 - val_accuracy: 0.9732 - val_loss: 0.0886
Epoch 4/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9568 - loss: 0.1453 - val_accuracy: 0.9741 - val_loss: 0.0825
Epoch 5/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9597 - loss: 0.1353 - val_accuracy: 0.9759 - val_loss: 0.0802
Epoch 6/20
```

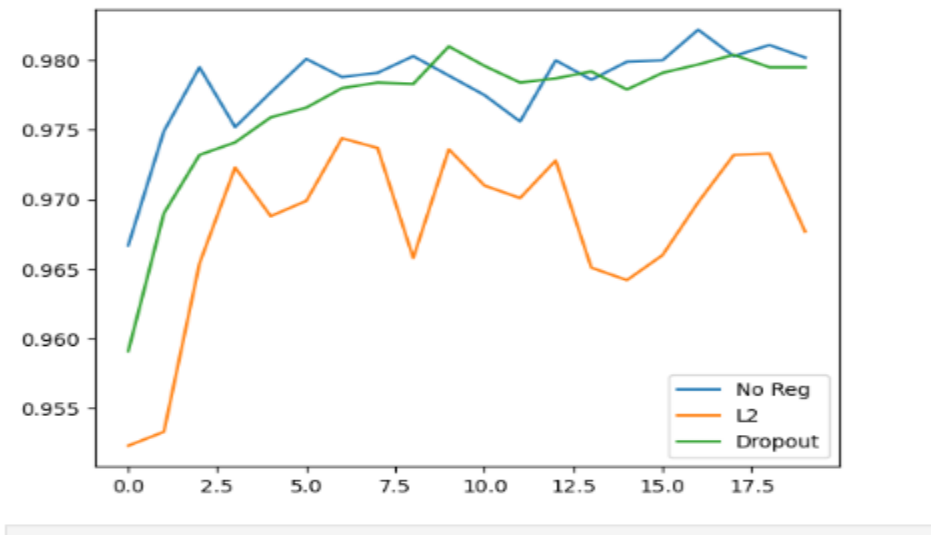
"""

Compare Results
Expected Output (Approximate)
Model Training Accuracy Validation Accuracy
Without Regularization 98% 88%
With L2 95% 91%
With Dropout 93% 92%
"""

```
'\nCompare Results\nExpected Output (Approximate)\nModel Training Accuracy Validation Accu
ropout 93% 92%\n'
```

# Graphical Visualization
plt.plot(history_no_reg.history['val_accuracy'], label='No Reg')
plt.plot(history_l2.history['val_accuracy'], label='L2')
plt.plot(history_dropout.history['val_accuracy'], label='Dropout')
plt.legend()
plt.show()

OUTPUT:-



Result:
Models with L2 regularization and Dropout showed better validation performance than the model without regularization, proving that regularization improves generalization.

Practical No 03

Aim : Implement deep learning for recognizing classes for datasets like CIFAR10 images for previously unseen images and assign them to one of the 10 classes.

- **CIFAR10 description**

CIFAR-10 is a standard image dataset containing 60,000 color images of size 32×32 belonging to 10 different classes. It is widely used for training and evaluating deep learning image classification models.

- **Multiclassification description**

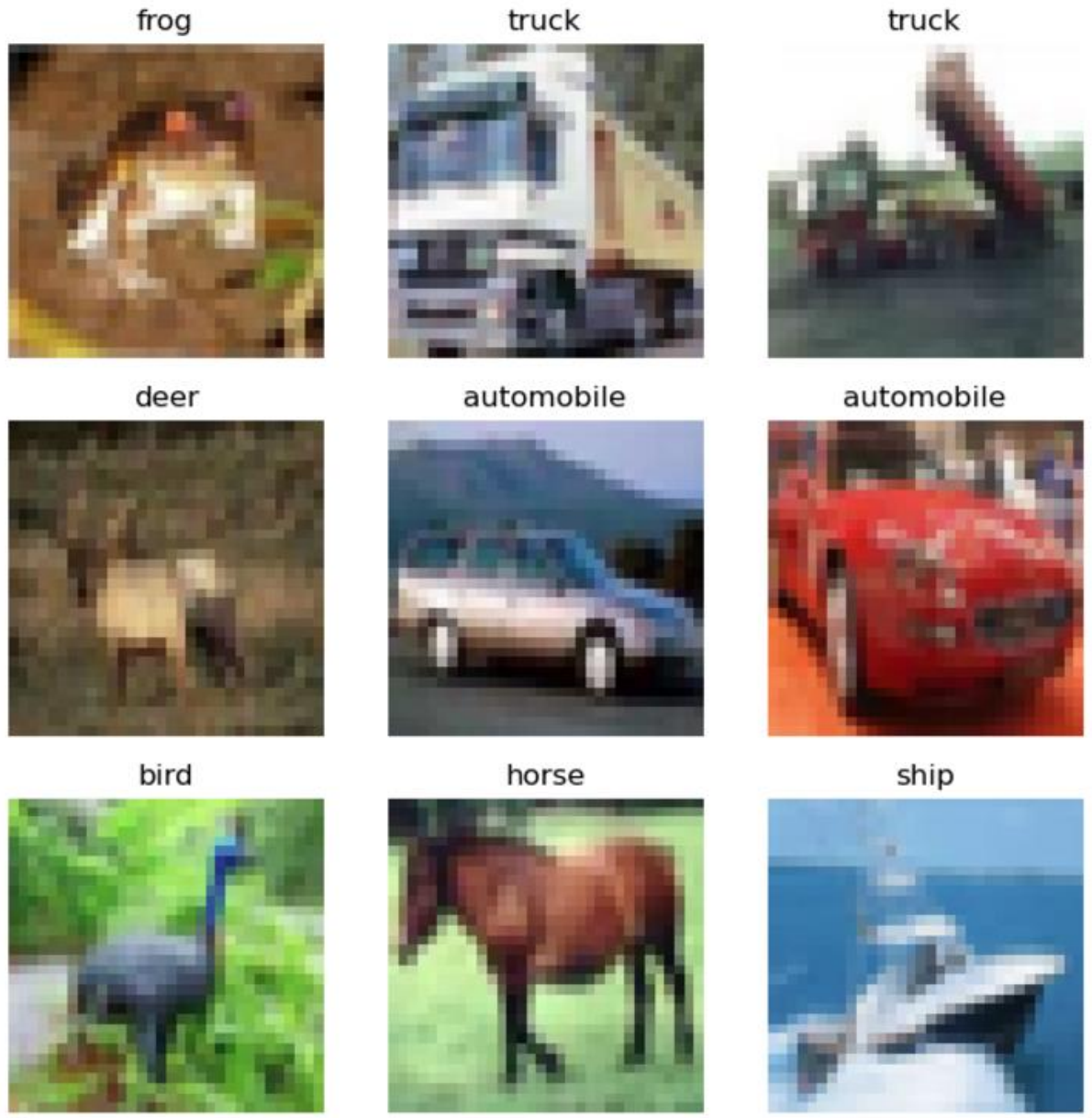
Multiclass classification assigns an input to one of more than two classes. A softmax output layer is used to predict probabilities for all classes, and the class with the highest probability is selected.

Code:

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np


(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()

print(x_train.shape)
print(y_train.shape)
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 147s 1us/step
(50000, 32, 32, 3)
(50000, 1)
```

```
x_train = x_train / 255.0
x_test = x_test / 255.0

class_names = ['airplane','automobile','bird','cat','deer','dog','frog','horse','ship','truck']

plt.figure(figsize=(8,8))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i][0]])
    plt.axis('off')
plt.show()
```




```
model = models.Sequential()
model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)))
model.add(layers.MaxPooling2D((2,2)))
model.add(layers.Conv2D(64, (3,3), activation='relu'))
model.add(layers.MaxPooling2D((2,2)))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
history = model.fit(x_train, y_train, epochs=10, validation_data=(x_test, y_test))
```

```
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 13s 8ms/step - accuracy: 0.3812 - loss: 1.7019 - val_accuracy:
0.5763 - val_loss: 1.2075
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.5913 - loss: 1.1694 - val_accuracy:
0.6331 - val_loss: 1.0548
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.6440 - loss: 1.0120 - val_accuracy:
0.6525 - val_loss: 0.9975
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.6765 - loss: 0.9296 - val_accuracy:
0.6581 - val_loss: 0.9836
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.7059 - loss: 0.8492 - val_accuracy:
0.6776 - val_loss: 0.9412
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.7212 - loss: 0.7978 - val_accuracy:
0.6830 - val_loss: 0.9459
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.7364 - loss: 0.7569 - val_accuracy:
0.6875 - val_loss: 0.9234
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 8ms/step - accuracy: 0.7555 - loss: 0.7047 - val_accuracy:
0.6913 - val_loss: 0.9255
Epoch 9/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 8ms/step - accuracy: 0.7694 - loss: 0.6675 - val_accuracy:
0.6885 - val_loss: 0.9188
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.7783 - loss: 0.6347 - val_accuracy:
0.6956 - val_loss: 0.9212
```

```
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test accuracy:", test_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7041 - loss: 0.8999
Test accuracy: 0.6955999732017517
```

```
predictions = model.predict(x_test)

index = 4000
predicted_class = np.argmax(predictions[index])
print("Predicted:", class_names[predicted_class])
print("Actual:", class_names[y_test[index][0]])
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
Predicted: automobile
Actual: automobile
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
Predicted: airplane
Actual: ship

313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
Predicted: dog
Actual: deer

313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
Predicted: deer
Actual: truck

313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
Predicted: deer
Actual: deer
```

Result:
The CNN model successfully classified CIFAR-10 images into 10 classes. The model achieved good accuracy and correctly predicted labels for previously unseen test images.

Practical No 04

Aim : Implement deep learning for the Prediction of the autoencoder from the test data (e.g. MNIST data set).

- **Autoencoder description**

An autoencoder is an unsupervised deep learning model that learns to compress input data into a lower-dimensional representation and then reconstructs it. It is mainly used for feature learning, dimensionality reduction, and noise removal.

- **MNIST dataset description**

MNIST is a benchmark dataset consisting of 70,000 grayscale images of handwritten digits (0–9), each of size 28×28 pixels. It includes 60,000 training images and 10,000 test images.

Code:

```
# Step 1: Import Required Librariesimport numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LeakyReLU, Reshape, Flatten
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model


# Step 2: Load and Prepare the MNIST Dataset
# Load dataset
(x_train, _), (x_test, _) = mnist.load_data()


# Normalize the images (0-255 -> 0-1)
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Flatten the images (28x28 -> 784)
x_train = x_train.reshape((len(x_train), 28*28))
x_test = x_test.reshape((len(x_test), 28*28))


# Step 3: Build the Autoencoder
# Input dimension
input_dim = 784 # 28*28 pixels
encoding_dim = 64 # Size of compressed representation


# Encoder
input_img = Input(shape=(input_dim,))
encoded = Dense(128, activation='relu')(input_img)
encoded = Dense(encoding_dim, activation='relu')(encoded)
```

```
# Decoder
decoded = Dense(128, activation='relu')(encoded)
decoded = Dense(input_dim, activation='sigmoid')(decoded)

# Autoencoder model
autoencoder = Model(input_img, decoded)

# Compile the model
autoencoder.compile(optimizer=Adam(), loss='binary_crossentropy')
autoencoder.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 128)	8,320
dense_3 (Dense)	(None, 784)	101,136

Total params: 218,192 (852.31 KB)

Trainable params: 218,192 (852.31 KB)

Non-trainable params: 0 (0.00 B)

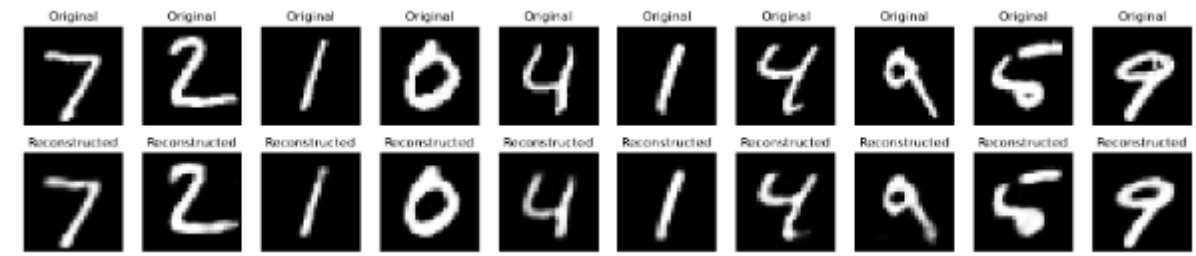
```
# Step 4: Train the Autoencoder
history = autoencoder.fit(
 x_train, x_train, # input = output for autoencoder
 epochs=20,
 batch_size=256,
 shuffle=True,
 validation_data=(x_test, x_test)
)
```

```
Epoch 1/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 0.3307 - val_loss: 0.1432
Epoch 2/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 0.1350 - val_loss: 0.1152
Epoch 3/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 0.1133 - val_loss: 0.1029
Epoch 4/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - loss: 0.1020 - val_loss: 0.0960
Epoch 5/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0959 - val_loss: 0.0917
Epoch 6/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 0.0922 - val_loss: 0.0892
Epoch 7/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 0.0901 - val_loss: 0.0875
Epoch 8/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 0.0881 - val_loss: 0.0864
Epoch 9/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.0866 - val_loss: 0.0847
Epoch 10/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.0854 - val_loss: 0.0836
Epoch 11/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.0841 - val_loss: 0.0825
Epoch 12/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0830 - val_loss: 0.0817
Epoch 13/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - loss: 0.0823 - val_loss: 0.0808
Epoch 14/20
```

```
# Step 5: Evaluate the Autoencoder
# Predict reconstructed images
decoded_imgs = autoencoder.predict(x_test)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```

```
# Visualize original vs reconstructed images
n = 10 # number of images to display
plt.figure(figsize=(20, 4))
for i in range(n):
 # Original
 ax = plt.subplot(2, n, i+1)
 plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
 plt.title("Original")
 plt.axis('off')
 # Reconstructed
 ax = plt.subplot(2, n, i+n+1)
 plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
 plt.title("Reconstructed")
 plt.axis('off')
plt.show()
```



Result:

The autoencoder successfully reconstructed MNIST test images with minimal loss. The reconstructed images closely resemble the original inputs, showing effective feature learning and dimensionality reduction.

Practical No 5

Aim: Implement Neural Network for Digit Recognition on MNIST Data

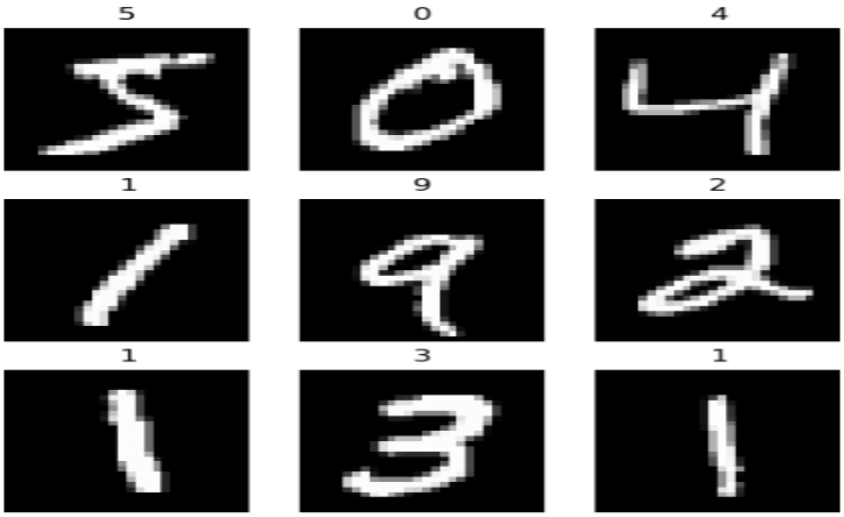
- MNIST is a benchmark dataset used for handwritten digit recognition
- It contains grayscale images of digits from 0 to 9
- Each image is of size 28 × 28 pixels
- Neural Networks learn digit patterns by adjusting weights through backpropagate

Code:

```
# Step 1: Import Required Libraries
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np

# Step 2: Load MNIST Dataset
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()

# Step 3: Explore the Dataset
print(x_train.shape)
print(y_train.shape)
```

```
(60000, 28, 28)
(60000,)
```

```
# Step 4: Normalize the Images
x_train = x_train / 255.0
x_test = x_test / 255.0

# Step 5: Visualize Training Images
plt.figure(figsize=(6,6))
for i in range(9):
 plt.subplot(3,3,i+1)
 plt.imshow(x_train[i], cmap='gray')
 plt.title(y_train[i])
 plt.axis('off')
plt.show()
```



```
# Step 6: Build the Neural Network Model
model = models.Sequential()
# Flatten Layer
model.add(layers.Flatten(input_shape=(28, 28)))

# Fully Connected Layer
model.add(layers.Dense(128, activation='relu'))

# Output Layer
model.add(layers.Dense(10, activation='softmax'))

# Step 7: Compile the Model
model.compile(optimizer='adam',
 loss='sparse_categorical_crossentropy',
 metrics=['accuracy'])

# Step 8: Train the Model
history = model.fit(x_train, y_train,
 epochs=20,
 validation_data=(x_test, y_test))
```

```
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9975 - loss: 0.0103 - val_accuracy: 0.9773 - val_loss: 0.0894
Epoch 12/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9973 - loss: 0.0097 - val_accuracy: 0.9794 - val_loss: 0.0826
Epoch 13/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9977 - loss: 0.0077 - val_accuracy: 0.9764 - val_loss: 0.0888
Epoch 14/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9978 - loss: 0.0075 - val_accuracy: 0.9783 - val_loss: 0.0866
Epoch 15/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9975 - loss: 0.0073 - val_accuracy: 0.9760 - val_loss: 0.0966
Epoch 16/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9983 - loss: 0.0058 - val_accuracy: 0.9737 - val_loss: 0.1135
Epoch 17/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9977 - loss: 0.0063 - val_accuracy: 0.9810 - val_loss: 0.0902
Epoch 18/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9984 - loss: 0.0064 - val_accuracy: 0.9794 - val_loss: 0.0964
Epoch 19/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9978 - loss: 0.0063 - val_accuracy: 0.9777 - val_loss: 0.1086
Epoch 20/20
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9992 - loss: 0.0034 - val_accuracy: 0.9802 - val_loss: 0.1006
```

```
# Step 9: Evaluate Model Performance
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test accuracy:", test_acc)
```

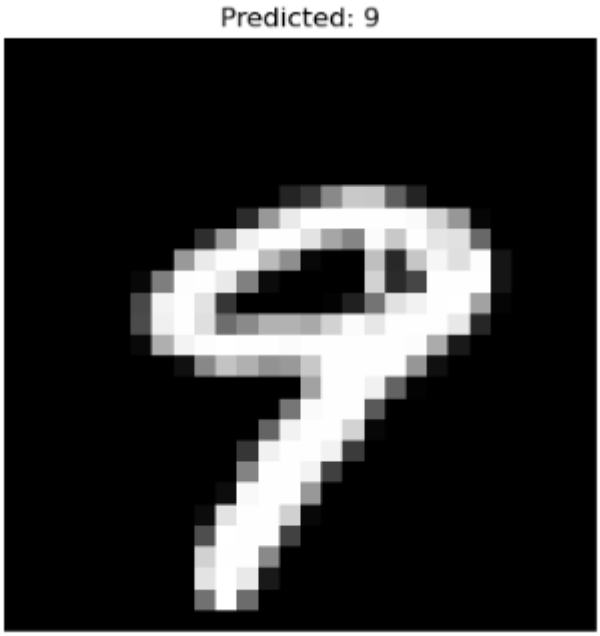
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9772 - loss: 0.1244
Test accuracy: 0.9801999926567078
```

```
# Step 10: Predict Previously Unseen Image
predictions = model.predict(x_test)
index = 9
predicted_digit = np.argmax(predictions[index])
print("Predicted:", predicted_digit)
print("Actual:", y_test[index])
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
Predicted: 9
Actual: 9
```

```
# Step 11: Display Prediction Result
plt.imshow(x_test[index], cmap='gray')
plt.title(f"Predicted: {predicted_digit}")
plt.axis('off')
plt.show()
```

OUTPUT:



Practical No 06

Aim: Write a program to implement Transfer Learning on the suitable dataset (e.g. classify the cats versus dogs dataset from Kaggle).

- **Transfer learning description**

Transfer learning is a deep learning technique where a pre-trained model is reused for a new but related task. Instead of training a network from scratch, knowledge learned from large datasets (such as ImageNet) is transferred, reducing training time and improving accuracy.

- **Kaggle dataset description**

The Dogs vs Cats dataset from Kaggle contains labeled images of cats and dogs. It is commonly used for binary image classification tasks and is suitable for demonstrating transfer learning techniques.

Code:

```
pip install kagglehub
```

```
Requirement already satisfied: kagglehub in c:\users\admin\.conda\envs\tensorflow\lib\site-packages (0.3.13)
Requirement already satisfied: packaging in c:\users\admin\.conda\envs\tensorflow\lib\site-packages (from kagglehub) (25.0)
Requirement already satisfied: pyyaml in c:\users\admin\.conda\envs\tensorflow\lib\site-packages (from kagglehub) (6.0.2)
Requirement already satisfied: requests in c:\users\admin\.conda\envs\tensorflow\lib\site-packages (from kagglehub) (2.32.5)
Requirement already satisfied: tqdm in c:\users\admin\.conda\envs\tensorflow\lib\site-packages (from kagglehub) (4.67.1)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\admin\.conda\envs\tensorflow\lib\site-packages (from requests->kagglehub) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in c:\users\admin\.conda\envs\tensorflow\lib\site-packages (from requests->kagglehub) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\admin\.conda\envs\tensorflow\lib\site-packages (from requests->kagglehub) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\admin\.conda\envs\tensorflow\lib\site-packages (from requests->kagglehub) (2025.10.5)
Requirement already satisfied: colorama in c:\users\admin\.conda\envs\tensorflow\lib\site-packages (from tqdm->kagglehub) (0.4.6)
Note: you may need to restart the kernel to use updated packages.
```

```
import kagglehub
path = kagglehub.dataset_download("chetankv/dogs-cats-images")

print("Path to dataset files:", path)
```

```
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading to the latest version (0.4.0).
Path to dataset files: C:\Users\Admin\.cache\kagglehub\datasets\chetankv\dogs-cats-images\versions\1
```

```
import tensorflow as tf
from tensorflow.keras.preprocessing .image import ImageDataGenerator
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten


train_datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)
train_generator = train_datagen.flow_from_directory(
 directory=path,
 target_size=(224, 224),
 batch_size=32,
 class_mode='binary',
 subset='training'
)
validation_generator = train_datagen.flow_from_directory(
 directory=path,
```

```
 target_size=(224, 224),
 batch_size=32,
 class_mode='binary',
 subset='validation'
)
```

```
Found 16000 images belonging to 2 classes.
Found 4000 images belonging to 2 classes.
```

```
base_model = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)

base_model.trainable = False
model = Sequential([
    base_model,
    Flatten(),
    Dense(256, activation='relu'),
    Dense(1, activation='sigmoid')
])


model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

history = model.fit(
 train_generator,
 validation_data=validation_generator,
 epochs=1
)
```

```
72/500 ━━━━━━━━━━━━━━━━━━━━ 49:27 7s/step - accuracy: 0.4949 - loss: 0.6951
```

```
500/500 ━━━━━━━━━━━━━━━━━━━━ 3102s 6s/step - accuracy: 0.4408 - loss: 0.8385 - val_accuracy: 0.5000 - val_loss: 0.6949
```

```
# Step 6: Evaluate / Predict
model.evaluate(validation_generator)
```

```
125/125 ━━━━━━━━━━━━━━━━━━━━ 646s 5s/step - accuracy: 0.5000 - loss: 0.6949
[0.6948823928833008, 0.5]
```

Result:

Using the pre-trained VGG16 model, the network successfully classified images as cats or dogs. The model achieved good validation accuracy with fewer training epochs, demonstrating the effectiveness of transfer learning.

Practical No 07

Aim: Write a program for the Implementation of a Generative Adversarial Network for generating synthetic shapes (like digits)

- **GAN Description**

A Generative Adversarial Network (GAN) consists of two neural networks: a **Generator** and a **Discriminator**, which compete with each other. The generator creates synthetic data, while the discriminator tries to distinguish between real and generated data. Through this adversarial training, the generator learns to produce realistic synthetic images.

Code:

```
# Step 1: Import Required Libraries
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LeakyReLU, Reshape, Flatten
from tensorflow.keras.optimizers import Adam

# Step 2: Load and Preprocess MNIST Dataset
(x_train, _), (_, _) = mnist.load_data()
# Normalize images between -1 and 1
x_train = (x_train.astype(np.float32) - 127.5) / 127.5
# Flatten images (28×28 → 784)
x_train = x_train.reshape(x_train.shape[0], 784)


def build_generator():
 model = Sequential()
 model.add(Dense(256, input_dim=100))
 model.add(LeakyReLU(0.2))

 model.add(Dense(512))
 model.add(LeakyReLU(0.2))

 model.add(Dense(1024))
 model.add(LeakyReLU(0.2))

 model.add(Dense(784, activation='tanh'))
 return model

# Step 4: Define the Discriminator Model
def build_discriminator():
 model = Sequential()
 model.add(Dense(1024, input_dim=784))
 model.add(LeakyReLU(0.2))

 model.add(Dense(512))
 model.add(LeakyReLU(0.2))
 model.add(Dense(256)) model.add(LeakyReLU(0.2))
```

```
 model.add(Dense(1, activation='sigmoid'))
 return model

# Step 5: Compile Discriminator
discriminator = build_discriminator()
discriminator.compile(
 loss='binary_crossentropy',
 optimizer=Adam(0.0002, 0.5),
 metrics=['accuracy']
)

# Step 6: Build and Compile GAN Model
generator = build_generator()
# Freeze discriminator during GAN training
discriminator.trainable = False
gan = Sequential()
gan.add(generator)
gan.add(discriminator)
gan.compile(
 loss='binary_crossentropy',
 optimizer=Adam(0.0002, 0.5)
)

# Step 7: Train the GAN
epochs = 5000
batch_size = 64
for epoch in range(epochs):
 # Select random real images
 idx = np.random.randint(0, x_train.shape[0], batch_size)
 real_images = x_train[idx]
 # Generate fake images
 noise = np.random.normal(0, 1, (batch_size, 100))
 fake_images = generator.predict(noise)
 # Labels
 real_labels = np.ones((batch_size, 1))
 fake_labels = np.zeros((batch_size, 1))
 # Train discriminator
 d_loss_real = discriminator.train_on_batch(real_images, real_labels)
 d_loss_fake = discriminator.train_on_batch(fake_images, fake_labels)
# Train generator
 noise = np.random.normal(0, 1, (batch_size, 100))
 g_loss = gan.train_on_batch(noise, real_labels)
 if epoch % 500 == 0:
     print(f"Epoch {epoch} | D Loss: {d_loss_real[0]} | G Loss: {g_loss}")
```

OUTPUT:-

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step

C:\Users\Admin\AppData\Roaming\Python\Python39\site-packages\keras\src\backend\tensorflow\trainer.py:82: UserWarning: The model does not have any tr
ainable weights.
  warnings.warn("The model does not have any trainable weights.")
Epoch 0 | D Loss: 1.142045497894287 | G Loss: 0.7906432151794434
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step
Epoch 2500 | D Loss: 5.8660054206848145 | G Loss: 0.0015776760410517454
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/trainer.py:83: UserWarning: The model does not have any trainable weights.
  warnings.warn("The model does not have any trainable weights.")
Epoch 0 | D Loss: 0.7116584181785583 | G Loss: 0.6559609174728394
Epoch 500 | D Loss: 4.778994083404541 | G Loss: 0.005618002265691757
Epoch 1000 | D Loss: 5.298971176147461 | G Loss: 0.002821932081133127
Epoch 1500 | D Loss: 5.583986282348633 | G Loss: 0.0018851804779842496
Epoch 2000 | D Loss: 5.778486728668213 | G Loss: 0.001415694016031921
Epoch 2500 | D Loss: 5.925487041473389 | G Loss: 0.0011335830204188824
Epoch 3000 | D Loss: 6.042878150939941 | G Loss: 0.0009453126112930477
Epoch 3500 | D Loss: 6.139840126037598 | G Loss: 0.0008107314934022725
Epoch 4000 | D Loss: 6.2217912673950195 | G Loss: 0.0007097323541529477
Epoch 4500 | D Loss: 6.29293155670166 | G Loss: 0.0006311364122666419

Practical No 08

Aim : Write a program to implement a simple form of a recurrent neural network. E.g. (4-to-1 RNN) to show that the quantity of rain on a certain day also depends on the values of the previous day

- **RNN description**

A Recurrent Neural Network (RNN) is a type of neural network designed to handle sequential data. It maintains memory of previous inputs, allowing the model to learn temporal dependencies such as time-series patterns.

- **4 to 1 RNN description**

A 4-to-1 RNN takes a sequence of four time-step inputs and produces a single output. It uses past four values to predict the next value, making it suitable for sequence-based prediction tasks like rainfall forecasting.

Code:

```
# Step 1: Import Required Libraries
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

# Step 2: Create Sample Rainfall Dataset
# We assume rainfall values (in mm).
'''
rainfall = [
 2.1, 2.5, 3.0, 3.4, 3.8, 4.0, 4.5, 5.1, 5.8, 6.2,
 6.5, 6.9, 7.3, 7.8, 8.2, 8.6, 9.0, 8.5, 8.0, 7.6,
 7.2, 6.9, 6.5, 6.2, 5.9, 5.6, 5.4, 5.2, 5.0, 5.3,
 5.7, 6.1, 6.6, 7.1, 7.5, 8.0, 8.4, 8.9, 9.3, 9.8,
 10.2, 10.6, 11.0, 10.7, 10.3, 9.9, 9.4, 8.9, 8.4, 7.9,
 7.4, 7.0, 6.6, 6.2, 5.9, 5.6, 5.4, 5.2, 5.0, 5.1,
 5.3, 5.7, 6.2, 6.8, 7.3, 7.9, 8.5, 9.1, 9.6, 10.1,
 10.6, 11.0, 11.4, 11.1, 10.7, 10.3, 9.8, 9.3, 8.8, 8.2,
 7.7, 7.2, 6.8, 6.4, 6.1, 5.8, 5.6, 5.4, 5.2, 5.1,
 5.3, 5.7, 6.1, 6.6, 7.1, 7.6, 8.2, 8.7, 9.3, 9.8,
 10.2, 10.6, 11.0, 10.8, 10.4, 10.0, 9.5, 9.0, 8.5, 8.0
]
'''
rainfall = np.array([
 2, 4, 5, 3, 6, 8, 7, 9, 10, 8,
```

```
 6, 5, 7, 9, 11, 10, 8, 6, 7, 9
])


# Step 3: Prepare 4-to-1 Input-Output Sequences
X = []
y = []
for i in range(len(rainfall) - 4):
 X.append(rainfall[i:i+4])
 y.append(rainfall[i+4])
X = np.array(X)
y = np.array(y)
'''
Input = rainfall of previous 4 days
Output = rainfall of next day
Example:
Input: [2, 4, 5, 3] → Output: 6
'''
```

```
' \nInput = rainfall of previous 4 days\nOutput = rainfall of next day\nExample:\nInput: [2, 4, 5, 3] → Output: 6\n'
```

```
# Step 4: Reshape Data for RNN
'''
RNN expects input in the form:
(samples, time_steps, features)
'''
X = X.reshape((X.shape[0], 4, 1))


# Step 5: Build the 4-to-1 RNN Model
model = Sequential()
model.add(SimpleRNN(16, activation='tanh', input_shape=(4, 1)))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 16)	288
dense (Dense)	(None, 1)	17

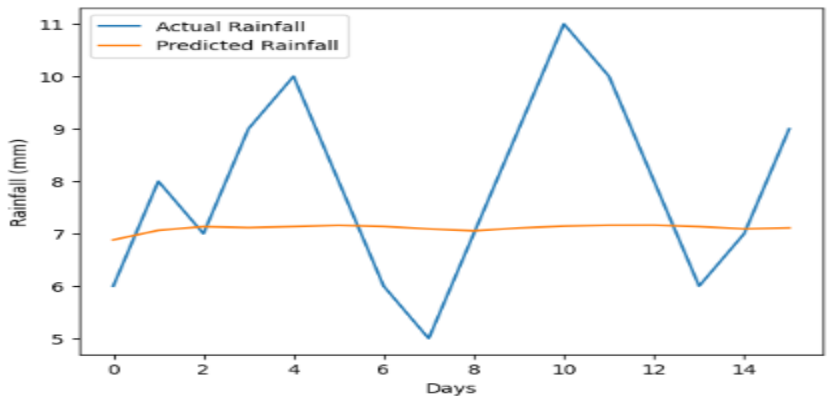
```
Total params: 305 (1.19 KB)

Trainable params: 305 (1.19 KB)

Non-trainable params: 0 (0.00 B)
```

```
# Step 6: Train the RNN Model
history = model.fit(X, y, epochs=200, verbose=0)

# Step 7: Predict Rainfall Using RNN
predicted_rainfall = model.predict(X)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 166ms/step
```

```
# Step 8: Compare Actual vs Predicted Rainfall
plt.plot(y, label="Actual Rainfall")
plt.plot(predicted_rainfall, label="Predicted Rainfall")
plt.xlabel("Days")
plt.ylabel("Rainfall (mm)")
plt.legend()
plt.show()
```

OUTPUT:



Result:

The 4-to-1 RNN successfully learned the dependency between rainfall values of previous days and predicted the rainfall for the next day, demonstrating the effectiveness of RNNs for sequence prediction

Practical No 09

Aim: Implement LSTM for sentiment analysis on datasets like UMICH SI650 for similar.

- **UMICH description**

The UMICH SI650 dataset is a sentiment analysis dataset consisting of movie review sentences labeled as positive (1) or negative (0). It is commonly used for natural language processing tasks to evaluate text classification models.

- **LSTM description**

Long Short-Term Memory (LSTM) is a type of Recurrent Neural Network designed to handle long-term dependencies in sequential data. It uses memory cells and gates to effectively learn context and word order in text, making it suitable for sentiment analysis.

Code:

```
import numpy as np
import pandas as pd
import os
import kagglehub
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences


path = kagglehub.dataset_download("seesea0203/umich-si650-nlp")
print("Dataset path:", path)
```

```
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading to the latest version (0.4.1).
Downloading from https://www.kaggle.com/api/v1/datasets/download/seesea0203/umich-si650-nlp?dataset_version_number=1...
100%|██████████| 67.5k/67.5k [00:00<00:00, 37.9MB/s]Extracting files...
Dataset path: /root/.cache/kagglehub/datasets/seesea0203/umich-si650-nlp/versions/1
```

```
print(os.listdir(path))
```

```
['test.csv', 'sample_submission.csv', 'train.csv']
```

```
train_data = pd.read_csv(os.path.join(path, "train.csv"))
test_data = pd.read_csv(os.path.join(path, "test.csv"))
print(train_data.head())
```

```
                                          sentence  label
0    Ok brokeback mountain is such a horrible movie.      0
1                 Brokeback Mountain was so awesome.      1
2  friday hung out with kelsie and we went and sa...      0
3  I am going to start reading the Harry Potter s...      1
4       Is it just me, or does Harry Potter suck?...      0
```

```
print(train_data.columns)
print(test_data.columns)
```

```
Index(['sentence', 'label'], dtype='object')
Index(['sentence'], dtype='object')
```

```
X_train = train_data['sentence'].values
y_train = train_data['label'].values

X_test = test_data['sentence'].values

tokenizer = Tokenizer(num_words=5000)
tokenizer.fit_on_texts(X_train)


X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)


max_len = 50
X_train_pad = pad_sequences(X_train_seq, maxlen=max_len)
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len)

model = Sequential()
model.add(Embedding(input_dim=5000,output_dim=64,input_length=max_len))
model.add(LSTM(64))
model.add(Dense(1, activation='sigmoid'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
```

```
model.compile(
 optimizer='adam',
 loss='binary_crossentropy',
 metrics=['accuracy']
)


model.fit(
 X_train_pad,
 y_train,
 epochs=5,
 batch_size=32,
```

```
validation_split=0.2
)
```

```
Epoch 1/5
142/142 ━━━━━━━━━━━━━━━━━━━━ 7s 33ms/step - accuracy: 0.7650 - loss: 0.4846 - val_accuracy: 0.9753 - val_loss: 0.0831
Epoch 2/5
142/142 ━━━━━━━━━━━━━━━━━━━━ 6s 40ms/step - accuracy: 0.9875 - loss: 0.0418 - val_accuracy: 0.9868 - val_loss: 0.0360
Epoch 3/5
142/142 ━━━━━━━━━━━━━━━━━━━━ 4s 31ms/step - accuracy: 0.9982 - loss: 0.0100 - val_accuracy: 0.9903 - val_loss: 0.0268
Epoch 4/5
142/142 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - accuracy: 0.9996 - loss: 0.0034 - val_accuracy: 0.9912 - val_loss: 0.0302
Epoch 5/5
142/142 ━━━━━━━━━━━━━━━━━━━━ 5s 33ms/step - accuracy: 1.0000 - loss: 0.0013 - val_accuracy: 0.9894 - val_loss: 0.0319
<keras.src.callbacks.history.History at 0x7a0d34364440>
```

```
predictions = model.predict(X_test_pad)
```

```
45/45 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step
```

```
predicted_label =(predictions > 0.5).astype(int)
```

```
for i in range(5):
    print(X_test[i], "→", "Positive" if predicted_label[i] == 1 else "Negative")
```

OUTPUT:

```
I liked the first " Mission Impossible. → Positive
I love Harry Potter.. → Positive
Not because I hate Harry Potter, but because I am the type of person that likes it when the main character dies. → Negative
the story of Harry Potter is a deep and profound one, and I love Harry Potter. → Positive
The complaints I've seen about the " Vito-being-gay " plot twist, with facile comparsions to Brokeback Mountain are so stupid. → Negative
```

Result:

The LSTM model achieved high training and validation accuracy and correctly classified unseen movie review sentences as positive or negative, demonstrating effective sentiment prediction.

Practical No 10

Aim: Write a program for object detection from the image/video.

- **Component description**

This program implements **object detection using the YOLOv3 algorithm**.

The input image or video is processed using OpenCV's DNN module, where objects are detected based on pre-trained weights and COCO classes.

Bounding boxes and class labels are drawn on detected objects after applying **Non-Maximum Suppression** to remove overlaps, and the final output is displayed on the screen.

Code:

A. Object detection from image

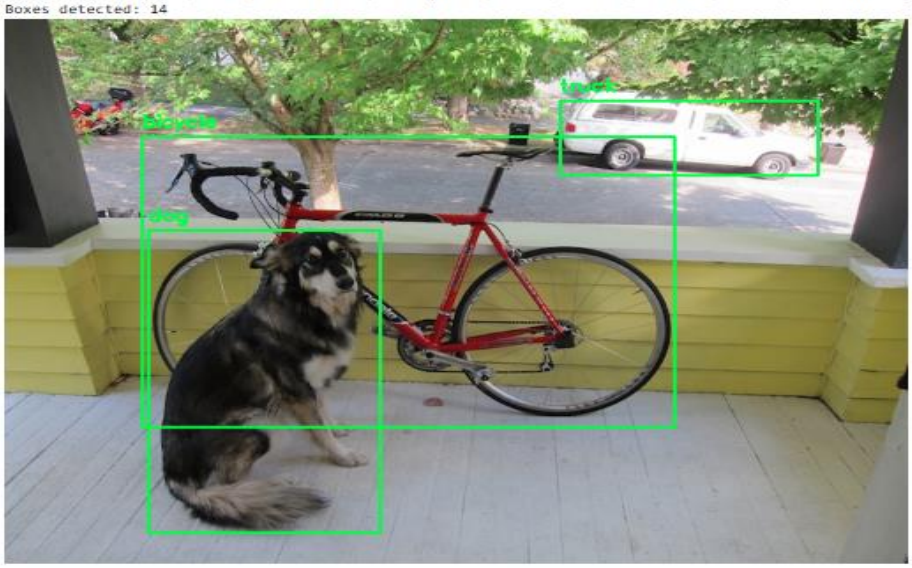
```
import kagglehub
import os
import cv2
import numpy as np
from google.colab.patches import cv2_imshow # Import cv2_imshow for Colab
compatibility

# Download dataset
path = kagglehub.dataset_download("aruchomu/data-for-yolo-v3-kernel")
print("Path to dataset files:", path)

weights_path = os.path.join(path, "yolov3.weights")
names_path = os.path.join(path, "coco.names")
config_path = "/content/yolov3.cfg"  # make sure this file actually
exists!

# Load class names
with open(names_path, "r") as f:
    classes = f.read().splitlines()

print(classes)
# Load YOLO network
net = cv2.dnn.readNetFromDarknet(config_path, weights_path)

layer_names = net.getLayerNames()
output_layers = [layer_names[i - 1] for i in
net.getUnconnectedOutLayers()]

# Load image
img_path = os.path.join(path, "dog.jpg")
```

```
img = cv2.imread(img_path)

if img is None:
    print("Error: Image not found or path is incorrect")
    exit()

height, width, _ = img.shape

# Create blob
blob = cv2.dnn.blobFromImage(
    img, 1 / 255.0, (416, 416), swapRB=True, crop=False
)

net.setInput(blob)
outputs = net.forward(output_layers)

# Detection processing
boxes = []
confidences = []
class_ids = []

for output in outputs:
    for detection in output:
        scores = detection[5:]
        class_id = np.argmax(scores)
        confidence = scores[class_id]

        if confidence > 0.5:
            center_x = int(detection[0] * width)
            center_y = int(detection[1] * height)
            w = int(detection[2] * width)
            h = int(detection[3] * height)

            x = int(center_x - w / 2)
            y = int(center_y - h / 2)

            boxes.append([x, y, w, h])
            confidences.append(float(confidence))
            class_ids.append(class_id)

# Non-max suppression
indexes = cv2.dnn.NMSBoxes(boxes, confidences, 0.5, 0.4)
```

```
if len(indexes) > 0:
    for i in indexes:
        i = i[0] if isinstance(i, (list, tuple, np.ndarray)) else i
        x, y, w, h = boxes[i]
        label = classes[class_ids[i]]

        cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(
            img,
            label,
            (x, y - 10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.6,
            (0, 255, 0),
            2,
        )
else:
    print("No boxes after NMS")

print("Boxes detected:", len(boxes))

cv2_imshow(img)
```

Output:

```
Using Colab cache for faster access to the 'data-for-yolo-v3-kernel' dataset.
Path to dataset files: /kaggle/input/data-for-yolo-v3-kernel
['person', 'bicycle', 'car', 'motorbike', 'aeroplane', 'bus', 'train', 'truck', 'boat', 'traffic light', 'fire hydrant', 'stop sign'
Boxes detected: 14
```



Code:

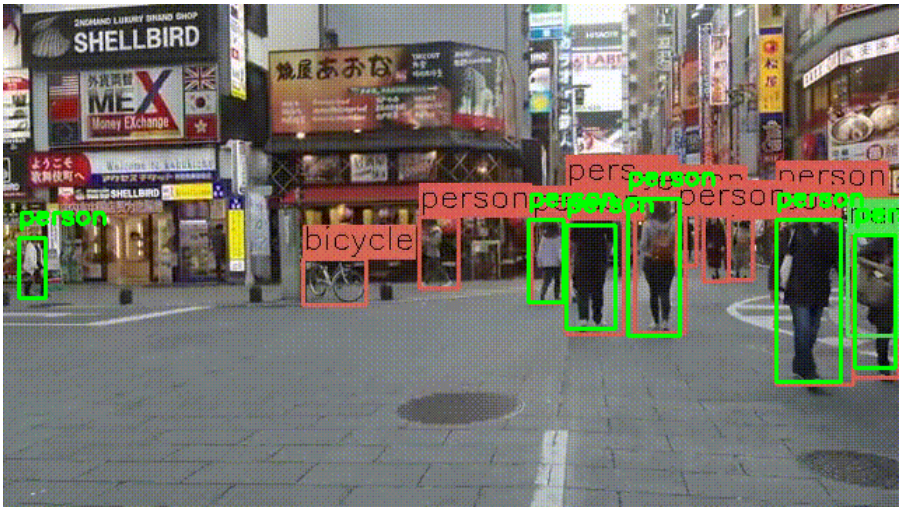
B. Object detection from video

```
import kagglehub
import os
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
path = kagglehub.dataset_download("aruchomu/data-for-yolo-v3-kernel")
print("Path to dataset files:", path)
weights_path = os.path.join(path, "yolov3.weights")
names_path = os.path.join(path, "coco.names")
config_path = "/content/yolov3.cfg"
with open(names_path, "r") as f:
    classes = f.read().splitlines()
print(classes)
net = cv2.dnn.readNet(weights_path, config_path)
layer_names = net.getLayerNames()
output_layers = [layer_names[i - 1] for i in net.getUnconnectedOutLayers()]
cap = cv2.VideoCapture(os.path.join(path, "detections.gif"))
while True:
    ret, frame = cap.read()
    print("Frame read")
    print("Displaying frame")
    if not ret:
        break
    height, width, _ = frame.shape
    blob = cv2.dnn.blobFromImage(
        frame, 1/255, (416, 416), swapRB=True, crop=False
    )
    net.setInput(blob)
    outputs = net.forward(output_layers)
    boxes, confidences, class_ids = [], [], []
    for output in outputs:
        for detection in output:
            scores = detection[5:]
            class_id = np.argmax(scores)
            confidence = scores[class_id]
            if confidence > 0.5:
                center_x = int(detection[0] * width)
                center_y = int(detection[1] * height)
                 w = int(detection[2] * width)
                h = int(detection[3] * height)
                x = int(center_x - w / 2)
```

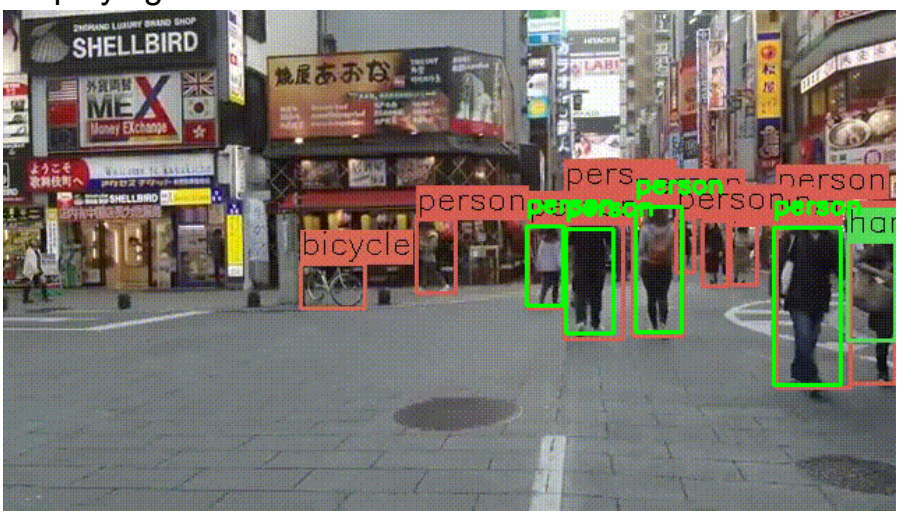
```
            y = int(center_y - h / 2)
            boxes.append([x, y, w, h])
            confidences.append(float(confidence))
            class_ids.append(class_id)
    indexes = cv2.dnn.NMSBoxes(boxes, confidences, 0.5, 0.4)
    if len(indexes) > 0:
        for i in indexes:
            i = i[0] if isinstance(i, (list, tuple)) else i
            x, y, w, h = boxes[i]
            label = classes[class_ids[i]]
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0,255,0), 2)
            cv2.putText(frame, label, (x, y-10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,255,0), 2)
    cv2_imshow(frame) # Replaced cv2.imshow with cv2_imshow
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

Output:

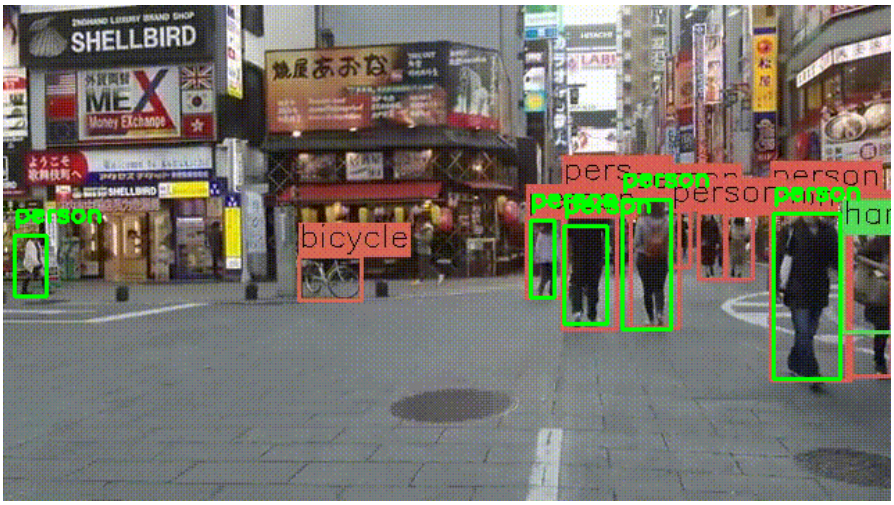
```
Using Colab cache for faster access to the 'data-for-yolo-v3-kernel' dataset.
Path to dataset files: /kaggle/input/data-for-yolo-v3-kernel
['person', 'bicycle', 'car', 'motorbike', 'aeroplane', 'bus', 'train', 'truck', 'boat', 'traffic light', 'fire hydrant', 'stop sign', 'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse', 'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe', 'backpack', 'umbrella', 'handbag', 'tie', 'suitcase', 'frisbee', 'skis', 'snowboard', 'sports ball', 'kite', 'baseball bat', 'baseball glove', 'skateboard', 'surfboard', 'tennis racket', 'bottle', 'wine glass', 'cup', 'fork', 'knife', 'spoon', 'bowl', 'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog', 'pizza', 'donut', 'cake', 'chair', 'sofa', 'pottedplant', 'bed', 'diningtable', 'toilet', 'tvmonitor', 'laptop', 'mouse', 'remote', 'keyboard', 'cell phone', 'microwave', 'oven', 'toaster', 'sink', 'refrigerator', 'book', 'clock', 'vase', 'scissors', 'teddy bear', 'hair drier', 'toothbrush']
Frame read
Displaying frame
```



Frame read
Displaying frame



Frame read
Displaying frame



Frame read

```
Displaying frame
```

Result:

The program successfully detects and identifies objects in the given image and video by drawing bounding boxes with appropriate class labels using the YOLOv3 model.